Crear un videojuego

Nombre: _____ Fecha: _____

Qué vas a aprender

- Identificar los elementos y las mecánicas de un videojuego
- Diseñar un juego en un documento de diseño antes de programarlo
- Programar el control del jugador, objetos con posiciones al azar y colisiones
- Usar variables para puntos, vidas y velocidad, y clones para multiplicar objetos
- Organizar el juego en pantallas con mensajes y ajustar la dificultad
- Probar, mejorar y compartir el juego pensando en la accesibilidad y la salud

Para pensar

¿Qué hace que un juego sencillo, como atrapar frutas que caen, sea divertido durante un rato y no aburrido ni imposible?

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

Rechazar Aceptar

Explicación

5.1 ¿Qué es un videojuego?

Un videojuego es un programa interactivo en el que un jugador intenta conseguir un objetivo siguiendo unas reglas. Aunque haya juegos muy distintos, casi todos comparten los mismos elementos:

- Un **jugador** que controla algo: un personaje, una nave, una cesta.
- Un **objetivo**: conseguir puntos, llegar a la meta, sobrevivir un tiempo.
- Unas **reglas**: qué se puede hacer y qué no, cómo se gana y cómo se pierde.
- Un **reto**: obstáculos, enemigos o un límite de tiempo que lo hacen difícil.
- **Retroalimentación**: el juego responde a lo que hace el jugador con sonidos, puntos o mensajes.
- **Recompensas**: puntos, niveles nuevos, sorpresas.

Las acciones básicas que el jugador repite (saltar, disparar, atrapar, esquivar) se llaman mecánicas. Un buen juego para empezar tiene **una sola mecánica** sencilla, pero bien hecha.



5.2 Géneros y mecánicas

Los videojuegos se clasifican en **géneros** según su mecánica principal. Algunos fáciles de programar en Scratch son:

- **Atrapar o esquivar**: objetos que caen y el jugador los recoge o los evita. Mecánica: moverse a izquierda y derecha.
- **Laberinto**: llegar a la salida sin tocar las paredes. Mecánica: moverse en cuatro

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

- **Preguntas** (trivial): responder bien para ganar puntos. Mecánica: preguntar y

comparar respuestas.
• **Carreras**: llegar antes que el rival, que puede ser otro jugador o el ordenador.
• **Naves**: esquivar asteroides y disparar.

Elegir el género ayuda a saber qué hay que programar. Antes de lanzarte, juega a algunos ejemplos en la web de Scratch y fíjate en qué hace cada objeto: es la mejor forma de aprender cómo están hechos.



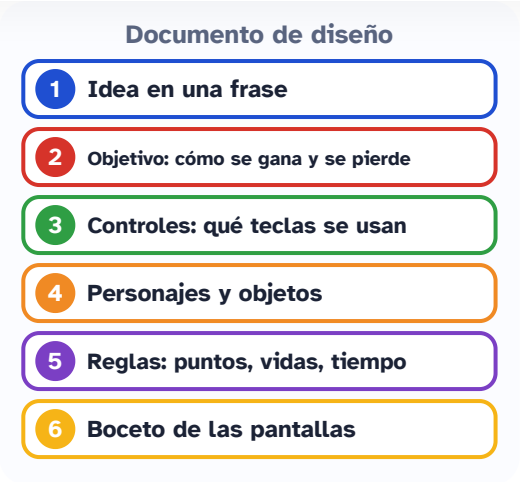
5.3 Diseñar antes de programar

Los estudios de videojuegos no empiezan programando: empiezan con un documento de diseño, una descripción del juego que todo el equipo puede consultar. Para un juego de clase basta con una página que responda a estas preguntas:

• **Idea**: ¿de qué va el juego, en una frase? «Atrapa la fruta con una cesta y esquiva las bombas».
• **Objetivo**: ¿cómo se gana y cómo se pierde? Ganar 50 puntos; perder al quedarse sin vidas.
• **Controles**: ¿qué teclas se usan? Flechas izquierda y derecha.
• **Personajes y objetos**: cesta, frutas, bomba, fondo.
• **Reglas**: +1 punto por fruta, –1 vida por fruta perdida o por bomba.
• **Pantallas**: portada, juego y fin.

Completa el diseño con un **boceto** en papel de cada pantalla. Después, divide el trabajo en partes (descomposición) y programa una cosa cada vez, probando cada parte antes de pasar a la siguiente.

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información



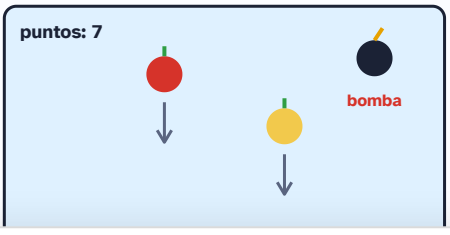
5.4 Nuestro proyecto: atrapar la fruta

Para aprender los conceptos del tema, construiremos un juego de **atrapar**. Su descripción es sencilla: frutas y bombas caen desde arriba en posiciones al azar; el jugador mueve una cesta con las flechas. Cada fruta atrapada suma un punto; cada fruta que llega al suelo o cada bomba atrapada resta una vida. Con `0` vidas, la partida termina.

Si descomponemos el juego, aparecen las partes que hay que programar:

1. Mover la **cesta** con el teclado sin que se salga de la pantalla.
2. Hacer que las **frutas caigan** desde un punto al azar.
3. Detectar las **colisiones** entre las frutas y la cesta.
4. Llevar la cuenta de **puntos y vidas**.
5. Tener varias frutas a la vez con **clones**.
6. Aumentar la **dificultad**.
7. Añadir **pantallas** de inicio y fin, **sonidos** y efectos.

Iremos programando cada parte en los apartados siguientes. Lo que aprendas te servirá para cualquier otro género.



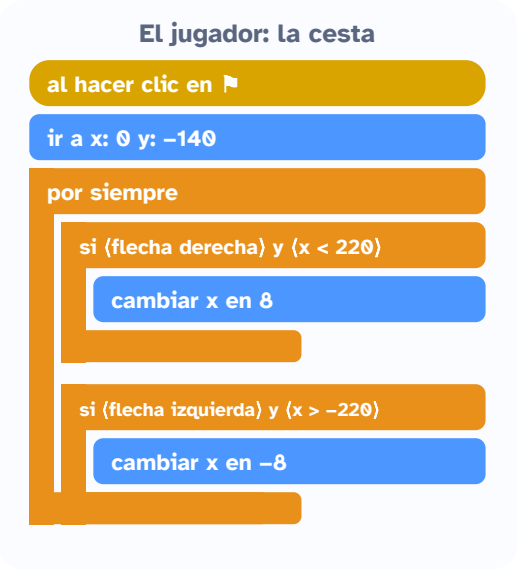
Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

5.5 Mover al jugador

El jugador debe responder rápido y con suavidad. Como viste con los bucles, la forma más fluida es un bucle «**por siempre**» con condicionales que comprueban las teclas: si la flecha derecha está presionada, cambiar x en 8; si la izquierda, cambiar x en −8.

Hay que evitar que la cesta **se salga de la pantalla**. Para ello, la condición se combina con un límite usando el operador «y»: solo se mueve a la derecha si la flecha está pulsada **y** la posición x es menor que 220. Otra opción es dejar que se mueva y, justo después, corregir: si x > 220, ir a x 220.

Al empezar, el jugador debe **colocarse** siempre en el mismo sitio con «ir a x: 0 y: −140». El número que se suma a x es la **velocidad** del jugador: si es muy pequeño, el juego será lento y frustrante; si es muy grande, será difícil de controlar. Prueba varios valores hasta encontrar el que se sienta bien.



5.6 Objetos que caen y el azar

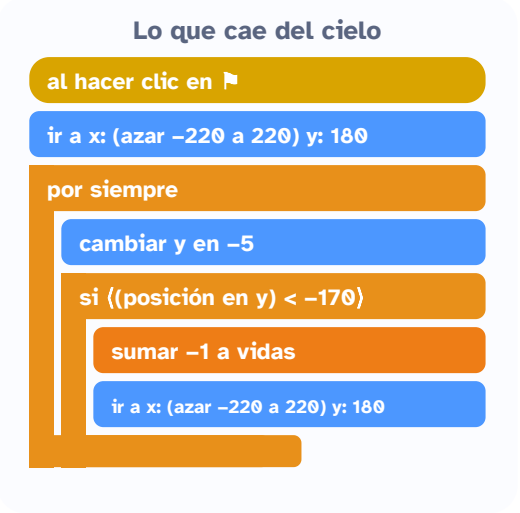
Si la fruta cayera siempre por el mismo sitio, el juego sería muy fácil. Para que sea imprevisible se usa el bloque de Operadores «**número aleatorio entre −220 y 220**», que da un número al azar distinto cada vez.

El programa de la fruta hace lo siguiente:

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

3. Si llega abajo (y < –170) sin ser atrapada, resta una vida y vuelve arriba, a otra x al azar.

El azar sirve para muchas más cosas en los juegos: elegir qué disfraz tendrá un objeto (manzana, pera o plátano), cuánto esperar antes de que aparezca el siguiente enemigo o qué pregunta saldrá en un trivial. Un poco de azar hace el juego más variado; demasiado azar hace que el jugador sienta que ganar o perder no depende de él.



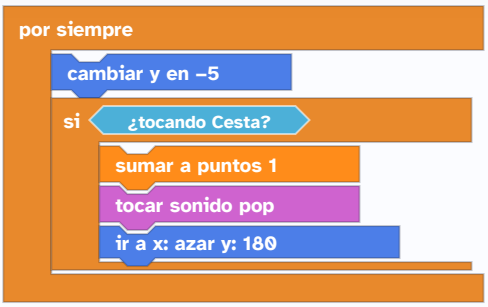
5.7 Las colisiones

Una colisión ocurre cuando dos objetos se tocan. Es el corazón de casi todos los juegos: atrapar, chocar, recoger, recibir un golpe. En Scratch se detecta con el sensor «**¿tocando ...?**», que puede comprobar si se toca otro objeto, el borde o el puntero del ratón. También existe «**¿tocando el color ...?**», muy útil en laberintos: si el personaje toca el color de las paredes, vuelve al inicio.

En nuestro juego, la fruta comprueba en su bucle: si ¿tocando Cesta?, entonces sumar 1 a puntos, iniciar el sonido «pop» y volver arriba a una x al azar. Es importante que, tras la colisión, la fruta **se aleje** enseguida; si no, en la siguiente vuelta del bucle seguiría tocando la cesta y sumaría puntos de más.

¿Quién debe comprobar la colisión, la fruta o la cesta? Normalmente, el objeto que **cambia** como resultado: la fruta, porque es ella la que desaparece y vuelve a caer.

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información



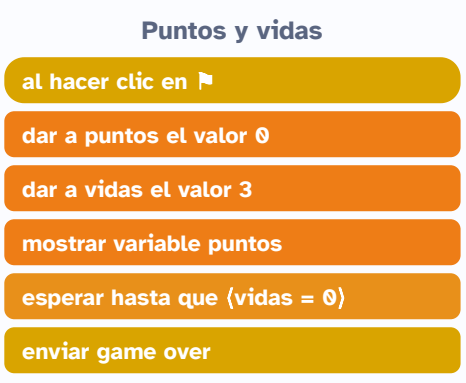
5.8 Variables: puntos y vidas

Para llevar la cuenta de lo que pasa en el juego se usan variables. En la categoría Variables se crea una con el botón «Crear una variable». Al crearla se elige si es **para todos los objetos** (global, como los puntos, que cambian varias frutas) o **solo para este objeto** (local, como la velocidad propia de cada enemigo).

Los bloques principales son:

- «**dar a puntos el valor 0**»: fija un valor. Se usa al principio para reiniciar la partida.
- «**sumar 1 a puntos**»: suma (o resta, con un número negativo).
- «**mostrar / esconder variable**»: el marcador en el escenario.

En nuestro juego hay dos: **puntos**, que empieza en 0, y **vidas**, que empieza en 3. Un programa del escenario espera hasta que vidas = 0 y entonces envía «game over». Recuerda inicializar siempre las variables al pulsar la bandera: si no, la nueva partida empezará con los puntos de la anterior.



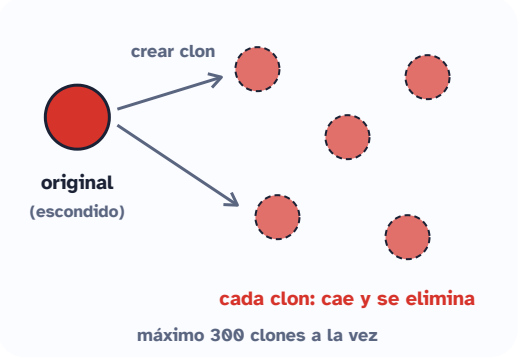
Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

Para que caigan muchas frutas a la vez podríamos duplicar el objeto fruta diez veces, pero si después hay que cambiar algo, habría que hacerlo en las diez copias. La solución son los clones: copias de un objeto que se crean **mientras el juego funciona** y que ejecutan el mismo programa.

Se usan tres bloques de Control:

• «**crear clon de mí mismo**»: el objeto original crea una copia.
• «**al comenzar como clon**»: evento que ejecuta cada clon nada más nacer (colocarse arriba, caer, comprobar colisiones).
• «**eliminar este clon**»: el clon desaparece cuando ya no hace falta, por ejemplo al llegar al suelo o al ser atrapado.

Un patrón típico: el original se esconde y, en un bucle, crea un clon cada 1 o 2 segundos; cada clon se muestra, cae y se elimina. Es importante **eliminar** los clones: Scratch permite como máximo 300 a la vez y, si se acumulan, el juego se vuelve lento.



5.10 Ajustar la dificultad

Un juego que siempre es igual de fácil aburre, y uno que es difícil desde el principio frustra. Los diseñadores buscan que la dificultad **crezca poco a poco**, a la vez que la habilidad del jugador. Algunas formas de conseguirlo:

• Usar una variable **velocidad** que aumenta cada cierto número de puntos: las frutas caen cada vez más rápido.
• Reducir el tiempo de espera entre clones: aparecen más frutas.
• Añadir obstáculos nuevos, como bombas, a partir de un nivel.

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

Para comprobar si se ha llegado a un múltiplo de 10 puntos se puede usar el operador «**módulo**» (resto de la división): si puntos mód 10 = 0, subir la velocidad. Ajustar estos números es un trabajo de prueba y error: juega, observa a otros y cambia los valores hasta que la partida sea un reto, pero un reto justo.

Cada vez más difícil

```
velocidad ← 3
por siempre
    cambiar y en (0 – velocidad)
    si toca Cesta entonces
        puntos ← puntos + 1
        si (puntos mód 10) = 0 entonces
            velocidad ← velocidad + 1
# cada 10 puntos, más rápido
```

5.11 Pantallas de inicio y de fin

Un juego completo no empieza de golpe ni se queda congelado al perder. Se organiza en **pantallas** o **estados**: la **portada** con el título y las instrucciones, la pantalla de **juego** y la de **fin** («game over» o «¡has ganado!») con la puntuación.

Una forma sencilla de programarlo es que el **escenario** dirija los estados con fondos y mensajes:

1. Al pulsar la bandera, fondo «Portada» y esperar hasta que se pulse espacio.
2. Fondo «Juego» y enviar «empezar»: los objetos, al recibirlo, se muestran y empiezan a funcionar.
3. Esperar hasta que vidas = 0, enviar «game over» y cambiar al fondo «Fin».
4. Al recibir «game over», los objetos se esconden y los clones se eliminan; se muestra la puntuación final.

Se puede añadir un récord con una variable «récord» que se actualiza si los puntos de la partida son mayores, y ofrecer volver a jugar con una tecla.

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información



5.12 Sonido, efectos y retroalimentación

La retroalimentación es la respuesta inmediata del juego a lo que hace el jugador. Sin ella, el jugador no sabe si ha hecho algo bien o mal. Los buenos juegos la dan por varios canales a la vez:

• **Sonido**: un «pop» al atrapar una fruta, un golpe al perder una vida, música que se acelera en los últimos segundos. Se usa «iniciar sonido» para no detener el juego.
• **Efectos visuales**: la cesta brilla un instante («cambiar efecto brillo»), el personaje parpadea al recibir un golpe, la pantalla tiembla.
• **Marcador**: puntos, vidas y tiempo siempre visibles.
• **Mensajes**: «¡Nivel 2!», «¡Nuevo récord!».

Estos detalles hacen que el juego se sienta «vivo» y agradable. Pero ojo: demasiados efectos a la vez cansan y pueden molestar a algunas personas. Ofrece una forma de bajar el volumen y evita los destellos rápidos de pantalla, que pueden ser peligrosos para personas con epilepsia fotosensible.



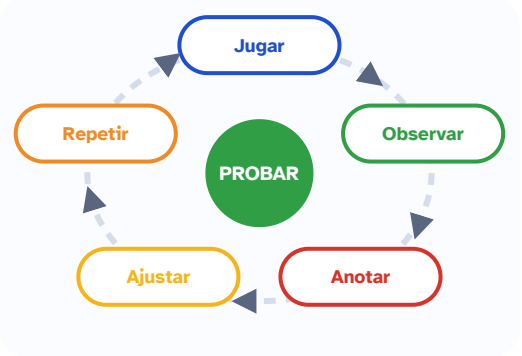
Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

Un juego no está terminado cuando funciona en tu ordenador: está terminado cuando **otras personas** lo juegan y lo disfrutan. Las pruebas con jugadores (en inglés, *playtesting*) son una parte esencial del desarrollo.

Cómo hacer una buena prueba:

• Deja jugar a un compañero **sin explicarle nada**: si no entiende qué hacer, faltan instrucciones.
• **Observa** en silencio: ¿dónde se atasca?, ¿se aburre?, ¿se ríe?
• **Anota** los problemas: errores (bugs), partes demasiado fáciles o difíciles, controles incómodos.
• **Ajusta** una cosa cada vez y vuelve a probar.

Revisa también los **casos raros**: ¿qué pasa si pulsas las dos flechas a la vez?, ¿y si pierdes justo al pasar de nivel?, ¿se reinicia todo bien al volver a jugar? Este ciclo de jugar, observar, anotar y ajustar es el mismo que usan los grandes estudios, solo que ellos lo repiten con miles de jugadores.



5.14 Videojuegos responsables y accesibles

Como creador de juegos, tienes en tus manos cómo se sentirán quienes los jueguen. Un juego accesible lo pueden disfrutar más personas: controles sencillos que se puedan cambiar, colores que también distingan las personas daltónicas (no uses solo rojo y verde para distinguir lo bueno de lo malo), letra grande y legible, subtítulos y opción de silenciar.

Un juego **saludable** respeta el tiempo de los jugadores: partidas cortas, pausa fácil y nada de trucos para enganchar, como premios que solo se consiguen

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

Por último, al compartir tu juego, respeta la autoría: si usas imágenes, música o código de otras personas, cítalas en los créditos. El sector de los videojuegos ofrece muchas profesiones: programación, diseño de niveles, arte, música, guion, pruebas de calidad o producción.



El ciclo de creación de un videojuego



Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

Ejercicios

1. **Nombra cuatro elementos que tiene casi todo videojuego.**

2. **¿Qué es una mecánica de juego? Pon tres ejemplos.**

3. **¿Qué debe incluir un documento de diseño sencillo?**

4. **¿Qué bloque da un número al azar en Scratch? ¿En qué categoría está?**

5. **¿Qué sensor se usa para detectar una colisión entre dos objetos?**

6. **¿Qué diferencia hay entre una variable para todos los objetos y una solo para este objeto?**

7. **Escribe los tres bloques principales para trabajar con clones.**

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

8. **¿Qué es la retroalimentación en un juego? Pon dos ejemplos.**

9. **¿Qué pantallas o estados suele tener un juego completo?**

10. **¿Qué indica el sistema PEGI?**

11. **Escribe el programa para que la cesta empiece en (0, –140) y se mueva con las flechas.**

12. **Una fruta se coloca en «ir a x: (aleatorio entre –220 y 220) y: 180». ¿Entre qué zonas puede aparecer?**

13. **Programa que, si la fruta toca la cesta, sume un punto, suene «pop» y vuelva arriba.**

14. **¿Por qué se usa «iniciar sonido» y no «tocar sonido hasta que termine» al atrapar una fruta?**

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

15. **Explica por qué conviene que sea la fruta, y no la cesta, la que compruebe la colisión.**

16. **Escribe el programa del escenario que pasa de la portada al juego al pulsar espacio.**

17. **¿Qué pasa si el objeto original crea clones en un bucle pero los clones nunca se eliminan?**

18. **Diseña las reglas de un juego de esquivar meteoritos: cómo se gana, cómo se pierde y qué da puntos.**

19. **Calcula: si la fruta baja 5 pasos por vuelta y el bucle da unas 30 vueltas por segundo, ¿cuánto tarda en caer de y = 180 a y = −170?**

20. **Propón tres formas de aumentar la dificultad de un juego de atrapar.**

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

22. **Escribe una lista de comprobación de cinco preguntas para probar un juego de un compañero.**

23. **Programa un laberinto: si el personaje toca el color de la pared, vuelve al inicio; si toca la salida, gana.**

24. **Programa la gravedad de un salto: velocidad vertical que se reduce hasta caer al suelo.**

25. **Diseña un sistema de niveles: a los 10 puntos nivel 2 con bombas; a los 25, nivel 3 con más velocidad.**

26. **Explica qué es el operador módulo y para qué sirve en un juego.**

27. **Un compañero dice que su juego es «perfecto» porque a él le funciona. ¿Qué le responderías?**

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

28. **Revisa la accesibilidad de un juego en el que las frutas son verdes y las bombas rojas. ¿Qué cambiarías?**

29. **Diseña el sistema de récord que muestre «¡Nuevo récord!» solo cuando se supere.**

30. **Escribe el documento de diseño completo de un juego original de una sola mecánica.**

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

Practica

1. **Las acciones básicas que el jugador repite se llaman...**

   A) variables
   B) mecánicas
   C) clones
   D) fondos

2. **Antes de programar un juego conviene hacer...**

   A) un documento de diseño
   B) los sonidos
   C) el récord
   D) la portada

3. **Para que la fruta aparezca en sitios distintos se usa...**

   A) ir a x: 0 y: 0
   B) rebotar
   C) detener todos
   D) número aleatorio

4. **Una colisión es cuando...**

   A) el juego se cuelga
   B) suena un sonido
   C) dos objetos se tocan
   D) se crea un clon

5. **Para detectar si se toca la pared de un laberinto se puede usar...**

   A) mostrar variable
   B) siguiente disfraz
   C) ¿tocando el color ...?
   D) crear clon

6. **Los puntos de un juego se guardan en...**

   A) un disfraz
   B) una variable
   C) un fondo

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

A) ponerse a 0
B) sumar 1
C) borrarse del proyecto
D) dividirse

8. **Una variable «para todos los objetos» es...**

A) local
B) un clon
C) un evento
D) global

9. **«crear clon de mí mismo»...**

A) duplica el proyecto
B) borra el objeto
C) guarda el juego
D) hace una copia del objeto mientras el juego funciona

10. **Los clones empiezan a funcionar con el evento...**

A) al hacer clic en ⚑
B) al recibir
C) al comenzar como clon
D) al presionar tecla

11. **Si no se eliminan los clones...**

A) desaparecen solos
B) se acumulan y el juego se ralentiza
C) suman puntos
D) cambian de color

12. **Si tras atrapar una fruta no se aleja de la cesta...**

A) suma puntos de más
B) se rompe el juego
C) pierde una vida
D) no pasa nada

13. **Para que la cesta no salga por la derecha se comprueba...**

A) que x sea menor que 220

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

D) el disfraz

14. **23 mód 10 es...**

A) 2

B) 2,3

C) 10

D) 3

15. **Una buena curva de dificultad...**

A) es difícil desde el principio

B) nunca cambia

C) empieza fácil y sube poco a poco

D) baja con el tiempo

16. **La pantalla de «game over» se suele activar con...**

A) un clon

B) un mensaje cuando vidas = 0

C) el lápiz

D) rebotar

17. **El sonido al atrapar una fruta es un ejemplo de...**

A) colisión

B) retroalimentación

C) variable

D) mecánica

18. **Probar el juego con otras personas se llama...**

A) playtesting

B) compilar

C) remezclar

D) clonar

19. **Para que un juego sea accesible a personas daltónicas...**

A) se usa solo rojo y verde

B) se quita el sonido

C) se hace más rápido

D) no se distinguen objetos solo por color

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

B) el precio

C) la edad recomendada de un juego

D) el género

Explícalo con tus palabras

Incluye: Qué elementos tiene un videojuego y qué es una mecánica; Cómo se diseña un juego antes de programarlo; Cómo se programan el movimiento, el azar y las colisiones; Para qué sirven las variables y los clones en un juego; Cómo se ajusta la dificultad y por qué hay que probar con otras personas.

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

Resumen



Glosario

Videojuego	Programa interactivo en el que un jugador busca un objetivo siguiendo unas reglas.
Mecánica	Acción básica que el jugador realiza repetidamente en un juego.
Género	Tipo de videojuego según su mecánica principal.
Documento de diseño	Descripción del juego con su idea, objetivo, controles, reglas y pantallas.
Azar	Resultado imprevisible que se obtiene con un número aleatorio.
Colisión	Contacto entre dos objetos del juego.
Variable global	Variable que pueden usar y cambiar todos los objetos.
Variable local	Variable propia de un solo objeto o de cada clon.
Clon	Copia de un objeto creada mientras el programa se ejecuta.
Dificultad	Grado de reto de un juego; suele aumentar con el progreso.

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

Módulo	Operación que da el resto de una división entera.
Retroalimentación	Respuesta inmediata del juego a las acciones del jugador.
Prueba de juego (playtesting)	Sesión en la que otras personas juegan para encontrar errores y mejoras.
PEGI	Sistema europeo que indica la edad recomendada de los videojuegos.
Accesibilidad	Característica de un juego que permite que lo disfruten personas con distintas capacidades.

Tarjetas de repaso

Recórtalas: pregunta delante, respuesta detrás.

Mecánica

Acción básica que repite el jugador.

Documento de diseño

Idea, objetivo, controles, reglas y bocetos.

Azar

número aleatorio entre … y …

Colisión

Dos objetos se tocan: ¿tocando …?

Variable global

Para todos los objetos: puntos.

Inicializar

Dar valor inicial al pulsar la bandera.

Clon

Copia creada durante el juego.

Eliminar clon

Evita que se acumulen (máx. 300).

Módulo

Resto de la división: detecta múltiplos.

Estados

Portada, juego y fin.

Retroalimentación

Respuesta inmediata: sonido, puntos,

Playtesting

Probar el juego con otras personas

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

Soluciones

1. **B)** Saltar, atrapar, esquivar...
2. **A)** Idea, reglas, controles, bocetos.
3. **D)** Azar.
4. **C)** Corazón de los juegos.
5. **C)** Detecta colores.
6. **B)** Valor que cambia.
7. **A)** Inicializar.
8. **D)** La comparten todos.
9. **D)** Clones en tiempo de ejecución.
10. **C)** Su propio sombrero.
11. **B)** Máximo 300.
12. **A)** Sigue tocando en la vuelta siguiente.
13. **A)** Límite del escenario.
14. **D)** Resto de la división.
15. **C)** Reto justo.
16. **B)** Estado de fin.
17. **B)** Respuesta del juego.
18. **A)** Prueba con jugadores.
19. **D)** Forma y sonido también.
20. **C)** Clasificación europea.

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

Soluciones de los ejercicios

1. Jugador, objetivo, reglas y reto (también retroalimentación y recompensas).
2. Una acción básica que el jugador repite. Ejemplos: saltar, atrapar y esquivar.
3. Idea en una frase, objetivo (ganar y perder), controles, personajes y objetos, reglas y boceto de las pantallas.
4. «número aleatorio entre … y …», en Operadores.
5. «¿tocando …?» de la categoría Sensores.
6. La global la pueden leer y cambiar todos los objetos (como los puntos); la local es propia de cada objeto (como su velocidad).
7. «crear clon de …», «al comenzar como clon» y «eliminar este clon».
8. La respuesta inmediata del juego a lo que hace el jugador. Ejemplos: un sonido al atrapar una fruta y el marcador de puntos.
9. Portada (con instrucciones), juego y fin (game over o victoria).
10. La edad recomendada de un videojuego según sus contenidos.
11. ⚑ · ir a x: 0 y: −140 · por siempre: si flecha derecha → cambiar x en 8; si flecha izquierda → cambiar x en −8.
12. En cualquier punto de la parte de arriba del escenario, de casi el borde izquierdo a casi el derecho.
13. si ¿tocando Cesta? entonces: sumar 1 a puntos, iniciar sonido Pop, ir a x: (aleatorio −220 a 220) y: 180.
14. Porque «tocar hasta que termine» detendría el bucle de la fruta mientras suena y el juego se congelaría un instante.
15. Porque la fruta es la que cambia tras la colisión (desaparece y vuelve arriba); si cada fruta lo comprueba, funciona igual aunque haya muchas.
16. ⚑ · cambiar fondo a Portada · esperar hasta que ¿tecla espacio presionada? · cambiar fondo a Juego · enviar empezar.
17. Se acumulan hasta el límite de 300, el juego va cada vez más lento y deja de crear clones nuevos.
18. Por ejemplo: +1 punto por cada segundo sobrevivido; −1 vida al tocar un meteorito; se pierde con 0 vidas; se gana al llegar a 60 puntos.
19. Recorre 350 pasos; a 5 por vuelta son 70 vueltas; a 30 vueltas por segundo, unos 2,3 segundos.
20. Aumentar la velocidad de caída, crear clones más a menudo y añadir bombas a partir de cierto número de puntos (o hacer la cesta más pequeña).
21. Al recibir «vida perdida»: repetir 3 veces: esconder, esperar 0,1 s, mostrar, esperar 0,1 s (o usar el efecto fantasma).
22. ¿Entiendo qué hacer sin ayuda? ¿Los controles responden bien? ¿Es demasiado fácil o difícil? ¿Hay algún bug? ¿Se reinicia bien al volver a jugar?

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

24. Al pulsar espacio y tocar el suelo: dar a vy el valor 12. Por siempre: cambiar y en vy; sumar −1 a vy; si toca el suelo y vy < 0: dar a vy el valor 0.
25. Variable nivel. Si puntos = 10 → nivel 2, enviar «nivel 2» (la bomba empieza a crear clones). Si puntos = 25 → nivel 3, sumar 2 a velocidad y mostrar «¡Nivel 3!».
26. Da el resto de una división: 23 mód 10 = 3. Sirve para detectar múltiplos (cada 10 puntos el resto es 0) o hacer ciclos, como alternar entre 3 disfraces.
27. Que debe probarlo con otras personas sin explicarles nada, porque el creador conoce los controles y los trucos. Los jugadores nuevos encuentran bugs y problemas de dificultad que él no ve.
28. Algunas personas daltónicas no distinguen bien el rojo del verde. Cambiaría también la forma (bomba redonda con mecha) y añadiría un sonido distinto.
29. Al recibir game over: si puntos > récord entonces dar a récord el valor puntos y decir «¡Nuevo récord!»; si no, decir «Récord: » unido a récord.
30. Respuesta abierta. Debe incluir idea en una frase, objetivo, controles, objetos, reglas de puntos y vidas, aumento de dificultad, pantallas y un boceto, y ser realizable en Scratch.

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información